BMS API + MCP server — specification for IT

Prepared 2026-10-19, on behalf of Brent Calaerts (Blitz Power). Intended to be forwarded to IT so they know what to build.

Executive summary

This document asks IT to build **two things** on top of your management system (referred to here as "BMS"):

1. **A read-only API** — a secured "gateway" that lets software **retrieve** data from BMS (never change it): charger logs and error messages, charging sessions/consumption, charger and status info, and customer/ticket info.
2. **An MCP server** on top of that API — a standard connector that lets an AI assistant such as **Claude query that data directly**. This way we build no client ourselves: Claude talks to the MCP server, the MCP server talks to the API, the API talks to BMS.

The goal: so that, on a service ticket, Claude can **fetch the right logs and info itself** (for triage and log analysis), instead of a human looking everything up and downloading it manually.

Key constraints:

- It stays **read-only** — the API must not change anything in BMS. Important: nothing can break.
- It touches **personal data** (customers). IT must keep this GDPR-compliant and access-controlled.
- IT fills in the BMS-specific parts (which system, which fields) — we only describe **what** we need, not how your system works internally.

---

1. Goal & context

Blitz Power handles service tickets about EV chargers. Today the charger log bundles (charX) and related info are looked up and downloaded **manually**. We want to automate this: during ticket triage, an AI assistant (Claude) should be able to **retrieve** the relevant logs and data **itself**.

This requires **programmatic access (an API)** to BMS, and an **MCP server** that exposes this access in an AI-friendly way.

This API is **independent of** the planning app; it is a separate, reusable service.

---

2. What we are asking for (overview)

#	Component	Short description
A	**Read-only REST API** on BMS	HTTPS API returning BMS data as JSON. No write operations.
B	**MCP server** on that API	A Model Context Protocol server exposing the API functions as "tools" to AI assistants (Claude).
C	**Documentation**	OpenAPI/Swagger for the API, and the connection URL + auth details for the MCP server.

Hosting: preferably in Blitz Power's existing **Azure environment** (subscription "Azure Blitz Power - BMS"), EU region (West Europe).

---

3. Scope

In scope (read only):

Domain	What we want to retrieve
Charger / charX logs & errors	charX log bundles (`.tar.gz`) per charger and period; list of errors/alarms/ERROR vectors; CP states.
Charging sessions & consumption	Sessions per charger/period/customer; session detail (kWh, duration, start/stop, connector, result).
Charger / station info & status	List of chargers/stations; detail (type, firmware, location); live **online/offline status**.

Domain	What we want to retrieve
Customer / contract / ticket info	Which customer/contract belongs to a charger or session; charger <-> customer mapping; ticket references.

Out of scope (for now):

- Anything that **writes** or **commands** (restart chargers, change settings, start/stop sessions). Explicitly excluded.
- Billing/payments.

---

4. The API — functional requirements (part A)

IT maps these functions onto the real BMS data. The names below are suggestions.

4.1 Endpoints (proposed)

Endpoint	Returns
`GET /v1/stations`	List of chargers/stations (filter by status, location, type).
`GET /v1/stations/{id}`	Detail of one charger (type, firmware, location, customer link).
`GET /v1/stations/{id}/status`	Live status (online/offline, last seen, active session).
`GET /v1/stations/{id}/logs?from=&to=`	List of available log bundles in a period.
`GET /v1/logs/{logId}`	Download of one charX log bundle (`.tar.gz`) or its contents.
`GET /v1/stations/{id}/errors?from=&to=`	Errors/alarms/ERROR vectors in a period.
`GET /v1/sessions?station=&from=&to=&customer=`	List of charging sessions with filters.
`GET /v1/sessions/{id}`	Session detail (kWh, duration, start/stop, connector, result).
`GET /v1/customers/{id}`	Customer/contract info (minimal, see privacy).
`GET /v1/search?q=`	Free search over charger, serial number, customer or ticket (useful for triage).

4.2 General conventions

- **Format:** JSON (log bundles as file/stream).
- **Time:** timestamps in ISO-8601 with time zone (or UTC), consistently.
- **IDs:** stable, unique identifiers per charger/session/customer.
- **Filtering & pagination:** by date, station, customer, status; pagination on large lists.
- **Errors:** clear HTTP status codes + JSON error message.
- **Versioning:** `/v1/` in the path, so later changes break nothing.
- **Documentation:** provide an **OpenAPI/Swagger** file.

---

5. Non-functional & technical requirements

Topic	Requirement
Read-only	The API has read rights on BMS only (least privilege). No write path whatsoever.
Transport	HTTPS only.
Authentication	Machine-to-machine: **OAuth2 client credentials via Microsoft Entra** (app registration) or an API key per consumer. Preference: Entra/OAuth.
Authorization	Access limited to authorized consumers; optionally scopes per domain (logs/sessions/customers separately).
Hosting	In Blitz Power's Azure environment, EU region. E.g. Azure App Service, Container Apps or Functions.
Rate limiting	To protect BMS.

Topic	Requirement
Logging/audit	All API access logged (who, when, what was requested) — especially for customer data.
Privacy (GDPR)	Keep personal data minimal; only necessary fields; access-controlled; retention/logging GDPR-compliant.
Availability	Must not overload BMS: consider a copy/cache or read replica rather than hitting the production system directly.

---

6. The MCP server (part B) — so we don't have to build a client

6.1 What it is (for IT)

An **MCP server** (Model Context Protocol, open standard — modelcontextprotocol.io) exposes the API functions as **"tools"** that an AI assistant such as Claude can call directly. Claude discovers the tools, calls them with parameters, and receives the result — without us writing any integration code.

6.2 Requirements

- **Type:** a **remote MCP server** (HTTP/streamable transport), reachable at a fixed HTTPS URL, hosted in the same Azure environment.
- **Build:** with the official MCP SDK (TypeScript or Python). The server is a thin layer: each tool calls the API (part A) internally.
- **Read-only tools**, with clear names and descriptions. Proposed:

Tool	Does	Parameters
`find_station`	Find a charger by name/serial/location	query
`station_info`	Return detail of a charger	station id
`station_status`	Return live online/offline status	station id
`list_logs`	List available log bundles	station id, period
`get_log`	Fetch/expose a charX log bundle	log id
`list_errors`	Errors/alarms in a period	station id, period
`list_sessions`	Charging sessions with filters	station/customer/period
`session_detail`	Detail of one session	session id
`find_customer`	Look up customer/contract info	query/id

- **Authentication of the MCP server:** preferably **OAuth**, so it can be added as a "connector" in Claude; an API-key header is an alternative.
- **Deliverables:** the MCP server's **connection URL** + how to connect it, plus the list of available tools.

---

7. Security & privacy (important)

- The API exposes **business and personal data**. Therefore: read-only, authenticated, EU-hosted, access-limited and logged.
- **Customer data** (domain in §3): minimize and restrict; only what triage needs.
- Store credentials (OAuth secret / API key) in **Azure Key Vault**, never share them loose.
- Access revocable per consumer.

---

8. What we need from IT (checklist)

1. **What is BMS concretely?** Which system/vendor, where does it run, who manages it?
2. **How do we reach the data?** Is there already an API/database access, or must it be exposed first?
3. **Data model:** which fields exist for chargers, sessions, logs, customers — and how do they relate (charger <-> customer <-> ticket)?
4. **charX logs:** where and how are the log bundles stored, and can they be retrieved per charger/period?

5. **Auth approach:** Entra/OAuth or API key? Who manages the app registration?
6. **Load:** may the API hit BMS directly, or is a copy/read replica better?
7. **Azure permissions:** who may create the resources (the planner test ran into this — see separate note)?
8. **Timeline & owner:** who builds and maintains this?

---

9. Recommended phasing

1. **Phase 1 — read the essentials:** stations + status + charX logs + errors (covers most of ticket triage).
2. **Phase 2 — sessions & consumption.**
3. **Phase 3 — customer/contract/ticket linking** (with privacy restrictions).
4. **Phase 4 — MCP server** on top of what exists by then, so Claude can use it.

Each phase delivers value immediately: once phase 1 exists, Claude can already fetch logs and status on a ticket.